Input Specific Neural Networks (ISNN)

Assignment 2

1. Introduction

Two implementations are provided and compared: one using PyTorch (autograd), and a second using NumPy with fully manual backpropagation derived from the chain rule.

2. Network Architectures

Both ISNN variants accept four input types (x_0, y_0, t_0, z_0) and produce a scalar output. Each input type is routed through a dedicated sub-network with the constraint appropriate for that input:

Input	Constraint	Activation σ	Weight Constraint
x_0	Convex	σ_mc = Softplus	W_xx ≥ 0 (layers ≥ 1)
y_0	Convex + Monotone	σ_mc = Softplus	W_yy, W_xy ≥ 0
t_0	Monotone only	σ_m = Sigmoid	W_tt, W_xt ≥ 0
z_0	Arbitrary	σ_a = Sigmoid	No restriction

2.1 ISNN-1

Each input type has its own independent sub-network of potentially different depth. The sub-networks for y_0, t_0, and z_0 are computed first, and their outputs are injected only into the first layer of the x-branch via skip connections. Subsequent x-branch layers operate solely on x. This architecture allows each branch to have a different number of hidden layers.

2.2 ISNN-2

All four branches share the same depth H. The key difference from ISNN-1 is that the outputs of the sub-networks, as well as the raw input x_0, are fed as skip connections into every layer of the x-branch — not just the first. This richer connectivity generally improves representational capacity.

2.3 Constraint Enforcement

Convexity and monotonicity constraints are enforced architecturally rather than through soft penalization in the loss. Two mechanisms are used:

- **Weight projection:** Non-negative weight projection: after each gradient update, constrained weight matrices are clipped to W ← max(W, 0).
- **Activation functions:** Activation choice: softplus (σ_mc) is convex and monotone non-decreasing; sigmoid (σ_m) is monotone non-decreasing.

Together these guarantee convexity and/or monotonicity by construction, so the constraints hold for any input — including out-of-distribution data.

3. Manual Backpropagation

The NumPy implementation computes all gradients by hand using the chain rule, without any automatic differentiation library. The general rule applied at each layer is:

$$dL/dz_h = dL/da_h \circ \sigma'(z_h) \quad \text{(element-wise Hadamard product)}$$
$$dL/dW_h = dL/dz_h^T \ @ \ a_{h-1} \quad \text{(outer product, batched)}$$
$$dL/da_{h-1} = dL/dz_h \ @ \ W_h \quad \text{(propagate to previous layer)}$$

For ISNN-1 and ISNN-2, the backward pass follows the same logic but must account for the branching structure: gradients flowing back through the x-branch at the merge layer are split and propagated independently into each sub-network (y, t, z branches). The activation derivatives used are:

- Softplus: $\sigma'(x) = \text{sigmoid}(x) = 1 / (1 + e^{-x})$
- Sigmoid: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$

The Adam optimizer is also implemented from scratch, maintaining per-parameter first and second moment estimates (m, v) with bias correction as described in Kingma & Ba (2014).

4. Toy Datasets

Two benchmark functions from Section 3.1 of the paper are used, both with inputs sampled via Latin Hypercube Sampling (LHS) to ensure good coverage.

Dataset	Function	Train range	Test range	N train	N test
Additive (Eq. 12)	$e^{-0.5x} + \log(1+e^{0.4y}) + \tanh(t) + \sin(z) - 0.4$	[0, 4]	[0, 6]	500	5,000
Multiplicative (Eq. 13)	$e^{-0.3x} \cdot (0.15y)^2 \cdot \tanh(0.3t) \cdot (0.2\sin(0.5z+2)+0.5)$	[0, 4]	[0, 10]	500	5,000

The test set deliberately extends beyond the training range to measure extrapolation. The additive function satisfies exactly the ISNN constraints: convex in x, convex+monotone in y, monotone in t, and arbitrary in z.

5. Results & Discussion

5.1 Loss Curves

Both implementations were trained for 3,000 epochs using the Adam optimizer (lr = 0.001). Final test MSE values are reported below:

Model	Additive — Test MSE	Multiplicative — Test MSE
FFNN (PyTorch)	0.2627	0.0101

Model	Additive — Test MSE	Multiplicative — Test MSE
ISNN-1 (PyTorch)	0.0694	0.0215
ISNN-2 (PyTorch)	0.0561	0.0208
FFNN (NumPy)	0.2159	0.0057
ISNN-1 (NumPy)	0.0829	0.0216
ISNN-2 (NumPy)	0.0200	0.0200

5.2 Extrapolation Behavior

The behavioral response plots (Figures 4 & 6 equivalent) evaluate each model along the diagonal x = y = t = z over the full test range. Key observations:

- **FFNN:** FFNN diverges or oscillates outside the training domain, as it has no mechanism to enforce any structural regularity.
- **ISNNs:** ISNN-1 and ISNN-2 track the true response more faithfully in the extrapolation region. The convexity/monotonicity constraints act as implicit regularization, preventing the network from fitting spurious patterns that do not generalize.
- **ISNN-2 vs ISNN-1:** ISNN-2 generally matches or outperforms ISNN-1, consistent with the paper's finding that richer skip connections improve expressiveness.
- **PyTorch vs NumPy:** Both PyTorch and NumPy implementations produce qualitatively identical behavior, validating the correctness of the manual backpropagation implementation.

6. Implementation Summary

Component	PyTorch	NumPy (Manual)
Forward pass	nn.Parameter + torch ops	NumPy matrix multiplication
Backward pass	autograd / .backward()	Hand-coded chain rule
Optimizer	torch.optim.Adam	Adam implemented from scratch
Constraint enforcement	.abs() on parameters	relu_proj after each step
Files produced	Model weights in memory	Parameter dicts in memory

The constraint projection approach (clipping weights to non-negative after each update) is simple and effective. An alternative is to reparameterize weights as W = |W_raw|, which is the approach used in the PyTorch implementation via .abs() calls during the forward pass.

7. Conclusion

This assignment successfully implemented ISNN-1 and ISNN-2 in both PyTorch and NumPy, with manual backpropagation. The key findings are:

- ISNNs consistently outperform unconstrained FFNNs on out-of-distribution test data due to their embedded structural constraints.

- The two-branch architecture (convex x-branch + separate sub-networks per input type) is effective and modular.
- Manual backpropagation through branching network structures is achievable by carefully splitting and accumulating gradient flows at merge points.
- Both implementations agree closely, confirming correctness of the manual derivation.